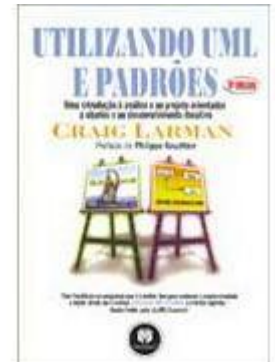


Utilizando UML e Padrões

Capítulo 17 – GRASP: Projeto de Objetos com Responsabilidades



Objetivos

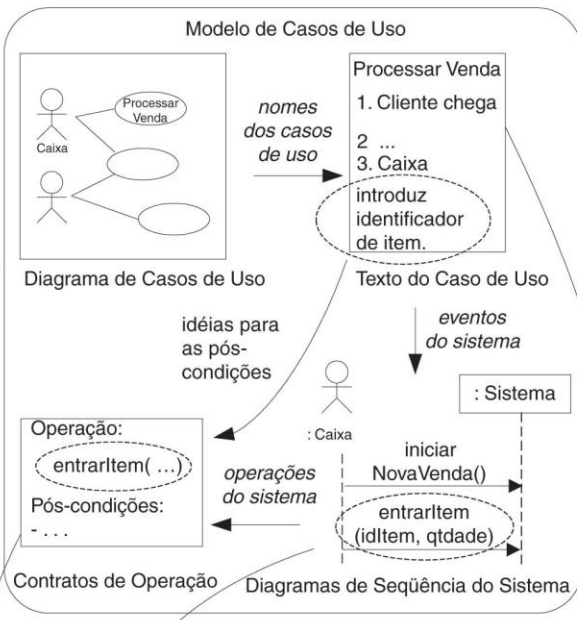
- Aprender a aplicar **cinco** dos **princípios** ou **padrões GRASP** para **POO**.

O que vem a seguir?



Diagrama de Modelo de Domínio (UML Class Diagram) mostrando a associação entre as classes **Venda** e **LinhaDeItemDeVenda**.

- A classe **Venda** possui o atributo **data**.
- A classe **LinhaDeItemDeVenda** possui o atributo **qtidade**.
- Existe uma associação entre **Venda** e **LinhaDeItemDeVenda** com multiplicidade **1** em **Venda** e **1..*** em **LinhaDeItemDeVenda**.
- Existem classes adicionais não nomeadas, representadas por retângulos com pontos suspensivos (...).



inspiração para
nomes de alguns
objetos do
domínio de
software

eventos iniciais
para os quais se
deve projetar e
pós-condições
detalhadas a
satisfazer

The diagram illustrates the interaction between three components: Registradora, CatálogoDeProdutos, and Venda.

Sequence Diagram:

- Participants:** Registradora, CatálogoDeProdutos, Venda.
- Start:** Registradora starts with a self-call `entrarItem(idItem, qtidade)`.
- Message 1:** Registradora sends `d = obterDescriçãoDoProduto(idItem)` to CatálogoDeProdutos.
- Message 2:** Registradora sends `adicionarLinhaDeItem(d, qtidade)` to Venda.

Class Diagram:

- Registradora Class:**
 - Operations: `iniciarNovaVenda()`, `entrarItem(...)`.
- CatálogoDeProdutos Class:**
 - Operation: `obterDescriçãoDoProduto`.
- Association:** Registradora (multiplicity `*`) is associated with CatálogoDeProdutos (multiplicity `1`).

requisitos funcionais
que precisam ser
realizados pelos
objetos.

regras de domínio

detalhes,
formatos e
validação
de item

Padrões GRASP

- **Projeto** orientado a objetos é:
 - Adicionar **métodos** em **objetos** presentes no **modelo de domínio**
 - Definir as **mensagens** entre **objetos**
 - quem colabora com quem
 - **Realizar os requisitos** através de um **sistema orientado a objetos**.

Padrões GRASP

- O **Projeto orientado a objetos** é uma atividade complexa
 - O melhor professor é a **experiência**.
 - Usando **padrões**, podemos aprender daqueles mais experientes.
 - É essencial entender **princípios de projeto** antes de fazer o **projeto orientado a objetos**.

Padrões GRASP

- **Projeto Dirigido por Responsabilidades - RDD**
 - Termo cunhado no início dos anos 90 por **Wirfs-Brock**
 - **Objetos** em termos do que eles **sabem** ou **fazem**
 - **metáfora** do trabalhador humano
 - Uma obrigação do objeto que é ofertado para outros objetos.
 - **Contrato**

Padrões GRASP

- Responsabilidades de “Fazer”
 - Criar outro **objeto**
 - Executar um **cálculo**
 - Iniciar uma ação em um **objeto**
 - Controlar/coordenar atividades de **objetos**.

Padrões GRASP

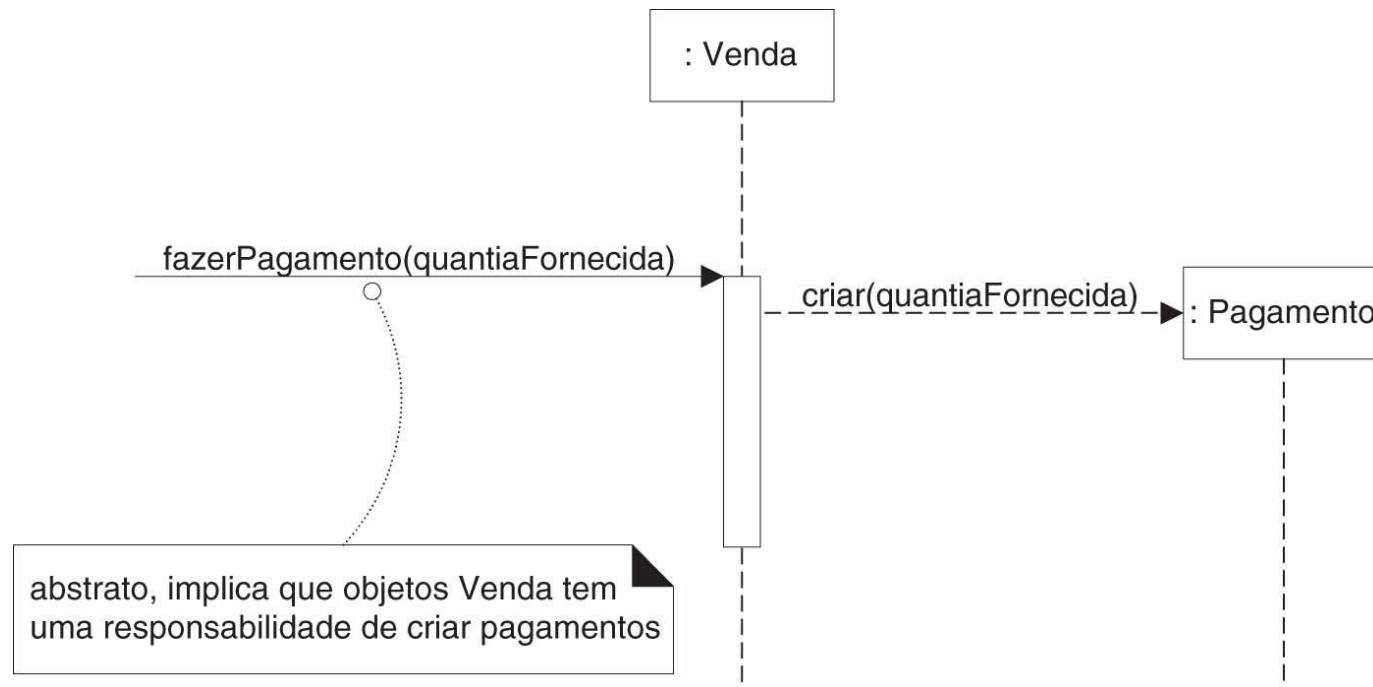
- Responsabilidades de “**Saber**”:
 - Seu próprio dado **encapsulado**
 - Sobre outros **objetos**
 - Coisas que ele pode **calcular** ou **derivar**.

Padrões GRASP

- **Granularidade da responsabilidade**
 - **Escala pequena**
 - Um **único objeto** preenche a responsabilidade
 - **Escala média**
 - Um **grupo de objetos** colaboram.
 - **Escala grande**
 - Um **sistema inteiro** ou subsistema necessário

Responsabilidades e métodos

- Alguém envia uma **mensagem** para **Venda** solicitando a **criação** de um **pagamento**.
 - Não significa que **Venda** cria um **Pagamento** diretamente



Padrões GRASP

- **Padrão**
 - Uma **solução nomeada** para **um problema recorrente**.
 - **Eficiência comprovada** por experiência anterior.
 - Elimina a **reinvenção da roda**
 - Facilita a **comunicação**
 - Indica **limitação** e **prós e contras**.

Padrões GRASP

- **GRASP**

- Do inglês “**General Responsibility Assignment Software Patterns**”
- Técnicas fundamentais para **Projeto dirigido por responsabilidades**.
- Devemos entender **GRASP** para fazer um bom **projeto orientado a objetos**.

Padrões GRASP

- **5 padrões neste capítulo**
 - **Criador**
 - **Especialista da Informação**
 - **Baixo acoplamento**
 - **Controlador**
 - **Alta coesão**

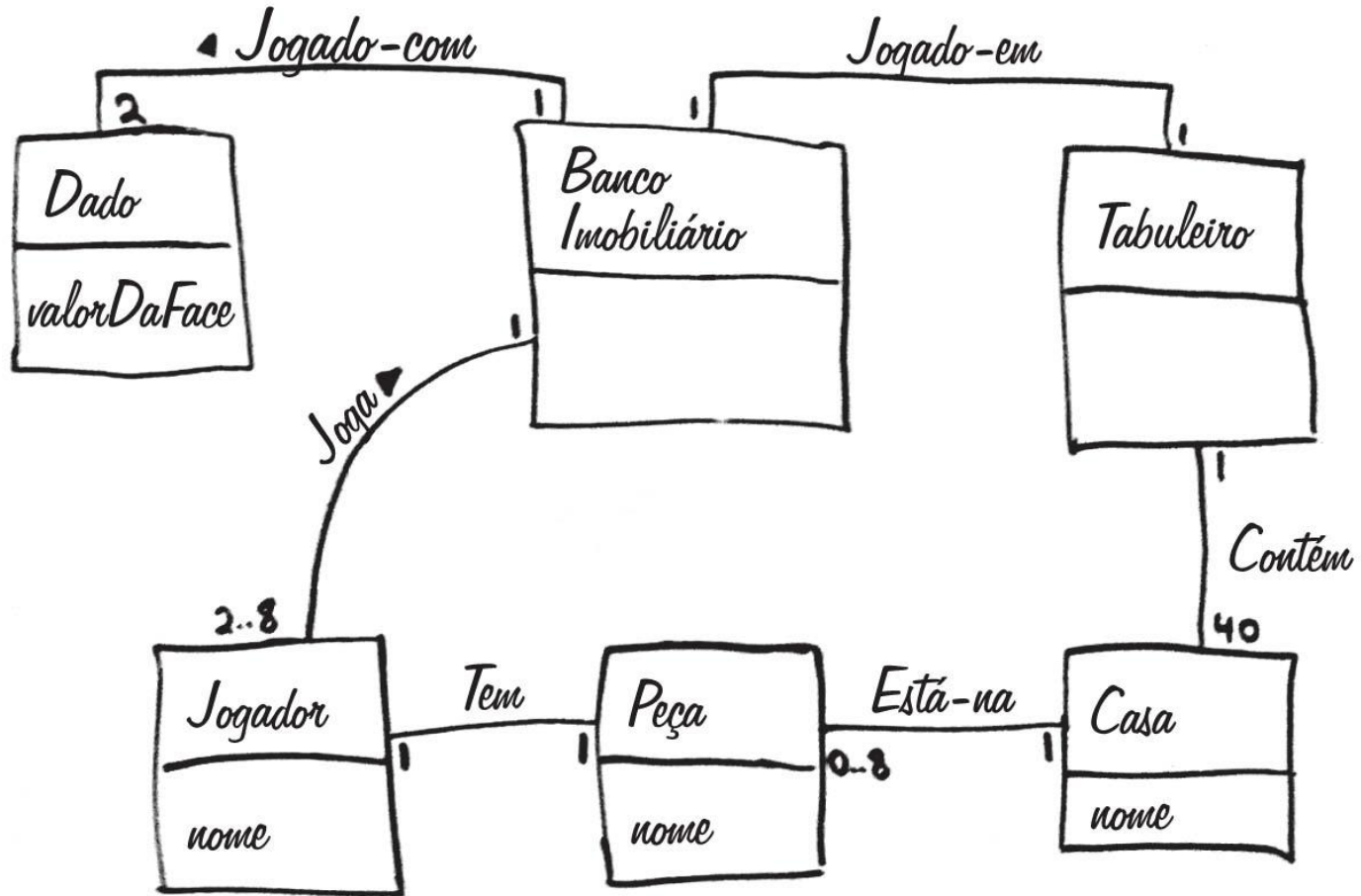
Padrões GRASP

- **Criador**
 - **Problema**
 - Quem deve criar um **objeto A**?
 - **Solução (sugestão)**
 - Deixe **B** fazer isso se:
 - **B** contém ou agrega **A**
 - **B** registra **A**
 - **B** usa **A** de maneira muito próxima
 - **B** contém os dados iniciais de **A**

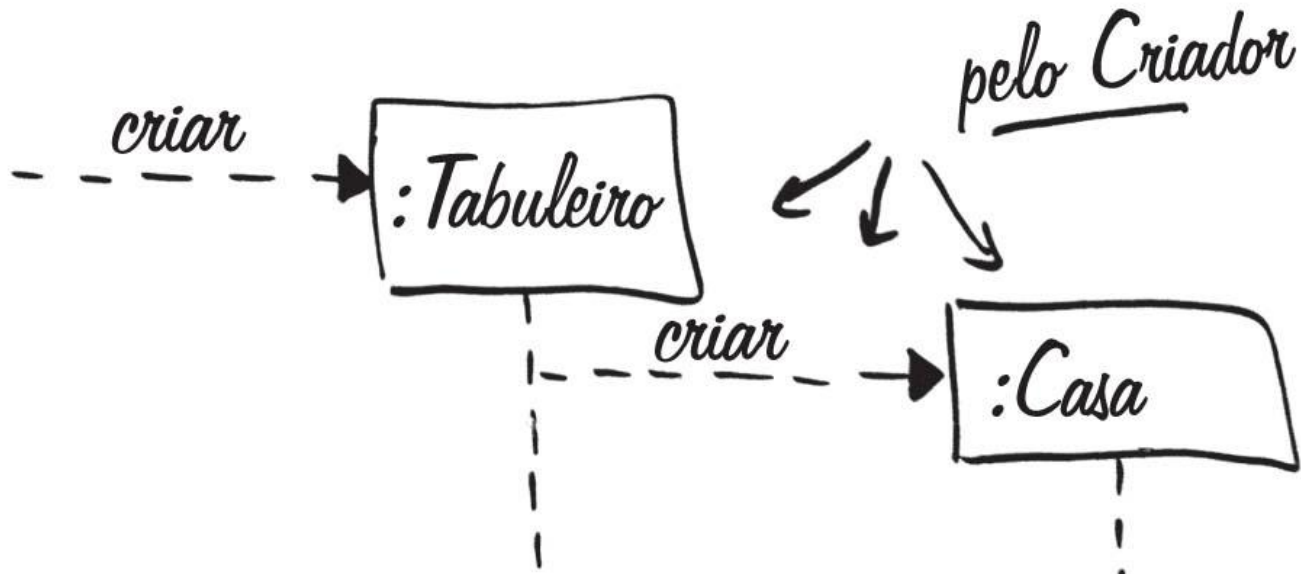
Padrões GRASP

- **Banco Imobiliário**
 - Quando você inicia o jogo, quem cria as **casas do tabuleiro**?
 - Deixe **Tabuleiro** criar já que é ele que contém as **casas**.

Banco Imobiliário



O Padrão Criador



Agregação Composta

- **Tabuleiro** possui uma relação de **composição** com Casa.



Padrões GRASP

- **Especialista da Informação**

- **Problema:**

- Qual o princípio básico de **Projeto Dirigido por Responsabilidades**?
 - Quem deveria executar uma tarefa?

- **Solução (conselho)**

- Atribua **responsabilidade** ao **objeto** que tem a informação necessária.

“Diga ao especialista para fazer”.

Padrões GRASP

- Banco Imobiliário

- Problema:

- Quem deveria obter (retornar) uma **Casa**, dado um determinado **identificador**?

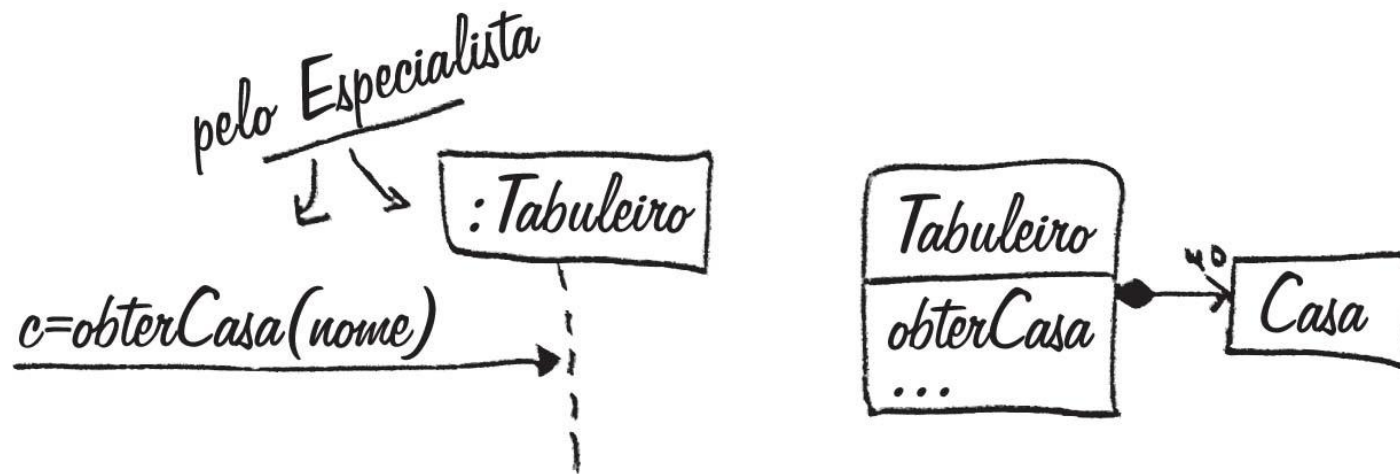
- Solução (Conselho):

- Deixe o **Tabuleiro** fazer isto pois ele sabe sobre as Casas.



O Padrão Especialista

- **Tabuleiro** contém uma **coleção de Casas**, portanto ele tem a **responsabilidade** de obter uma **Casa** em particular.



Padrões GRASP

- **Acoplamento Baixo**

- **Problema:**

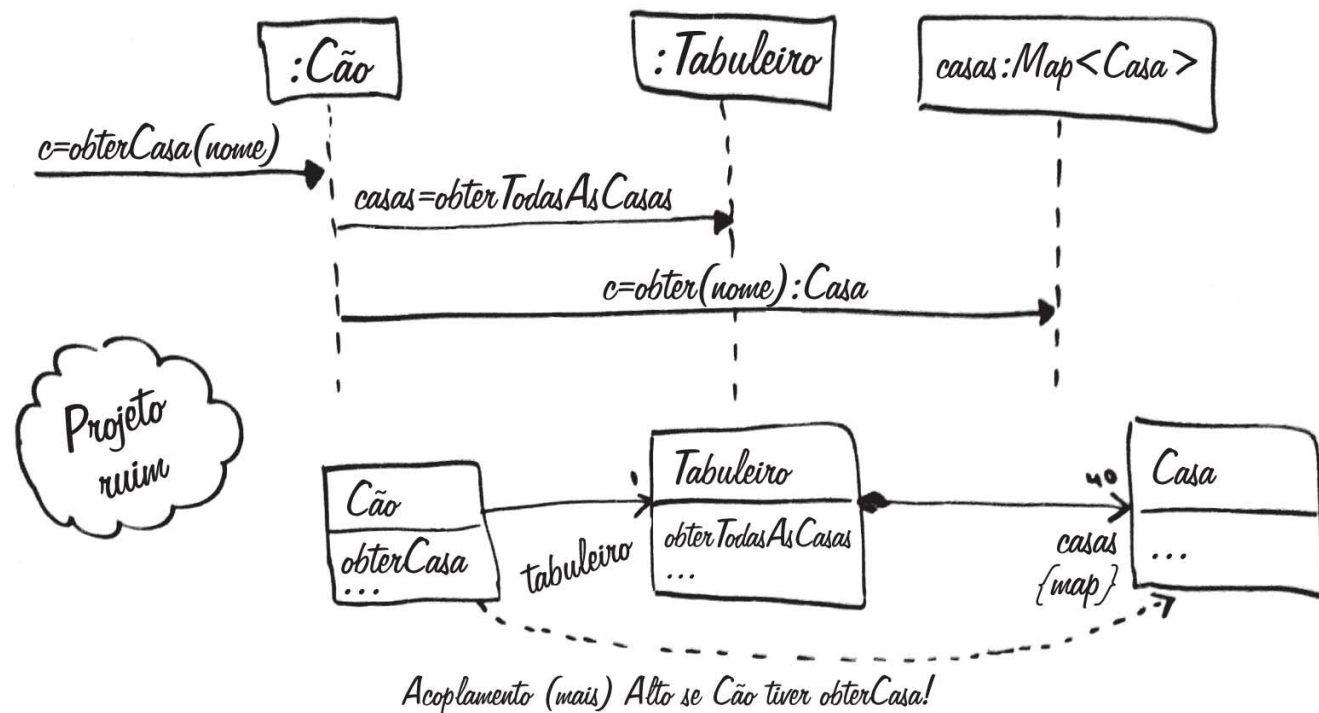
- Como reduzir o impacto de **mudança**?

- **Solução (conselho):**

- Atribuir **responsabilidade** para minimizar o **acoplamento** entre **objetos**.
 - **Avaliar** alternativas de projeto
 - Escolher a opção que **minimiza o acoplamento**

O Padrão Acoplamento Baixo

- Se **Cachorro** obter a **Casa**, então **Cachorro** e **Tabuleiro** ficam acoplados à **Casa** !



Padrões GRASP

- **Controlador**

- **Problema:**

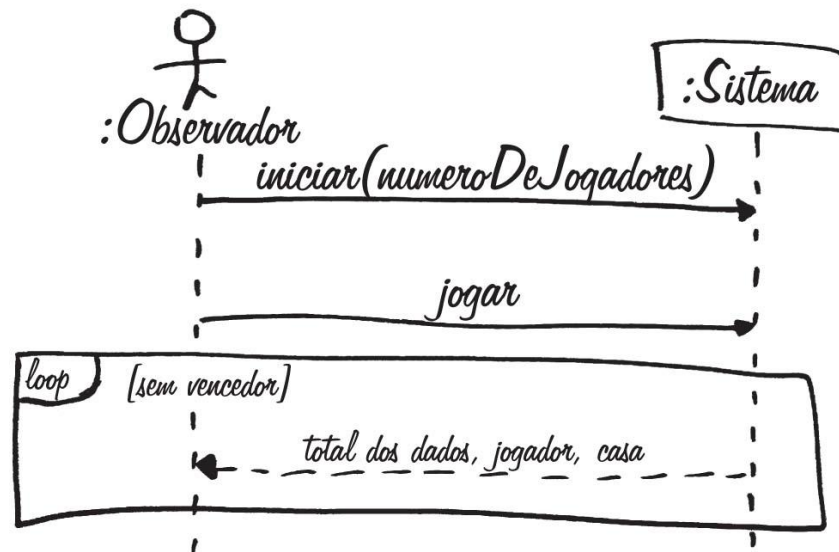
- As **camadas GUI e Domínio** devem ser **fracamente acopladas**
 - Que objeto deve **coordenar mensagens** entre a **GUI** e outros **objetos do domínio**?

- **Solução (conselho):**

- Atribua a **responsabilidade** a um objeto representando:
 - O “**sistema**” ou um objeto “**raiz**”
 - » **Loja, Banco**
 - Um dispositivo/subsistema
 - » **SistemaContabilidade, CaixaEletronico**
 - Um cenário de **caso de uso**
 - » **TratadorJogarBancoImobiliario**

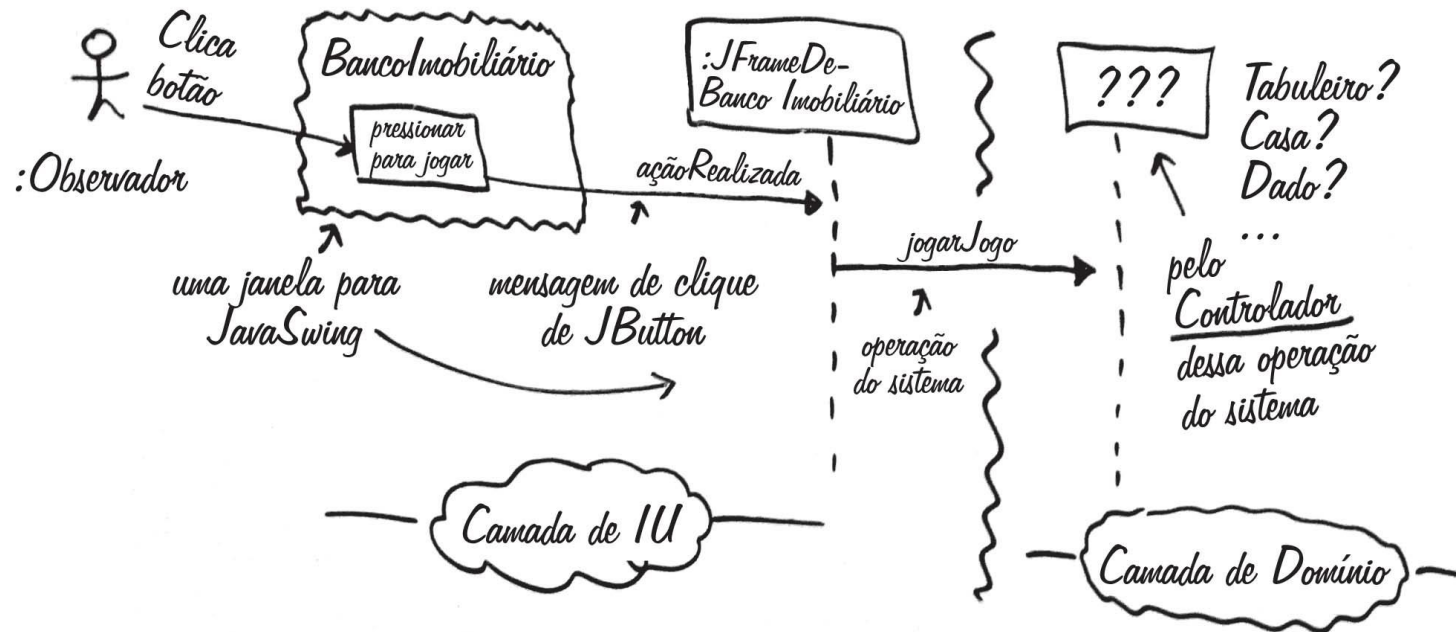
DSS para o jogo Banco Imobiliário

- O **usuário** interage diretamente com a **GUI** e não com a **camada de domínio**
- Que objeto deve retransmitir **operações do sistema** da **GUI** para a **camada de Domínio**?



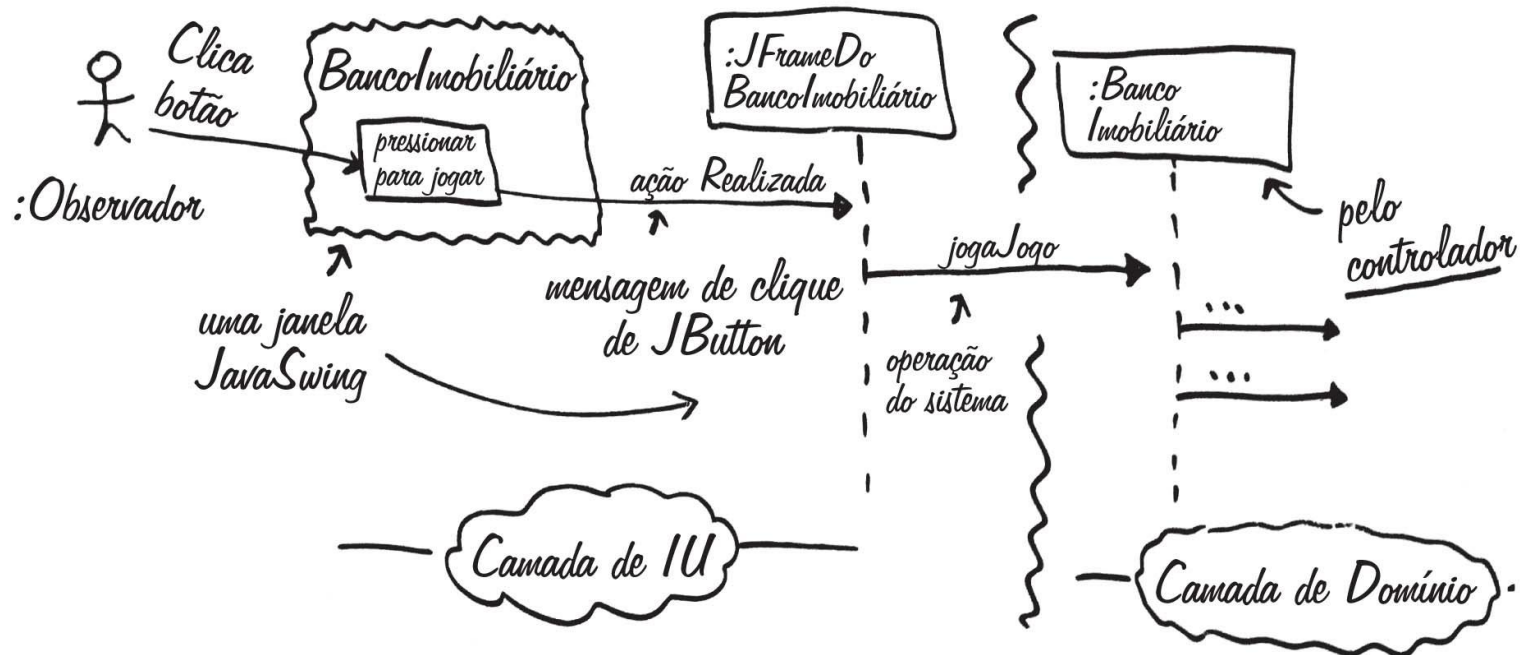
Visão em Camadas

- Tradução entre **eventos GUI** em **operações do sistema**
- Quem é o **mediador** entre a **GUI** e a **camada de domínio**?



O Padrão Controlador

- Deixe **BancoImobiliario** ser o controlador
- Ele representa o sistema e não existem muitas operações do sistema.



Padrões GRASP

- **Coesão Alta**

- **Problema:**

- Como manter os **objetos focados**, inteligíveis e gerenciáveis?
 - Como apoiar o **Baixo Acoplamento**?

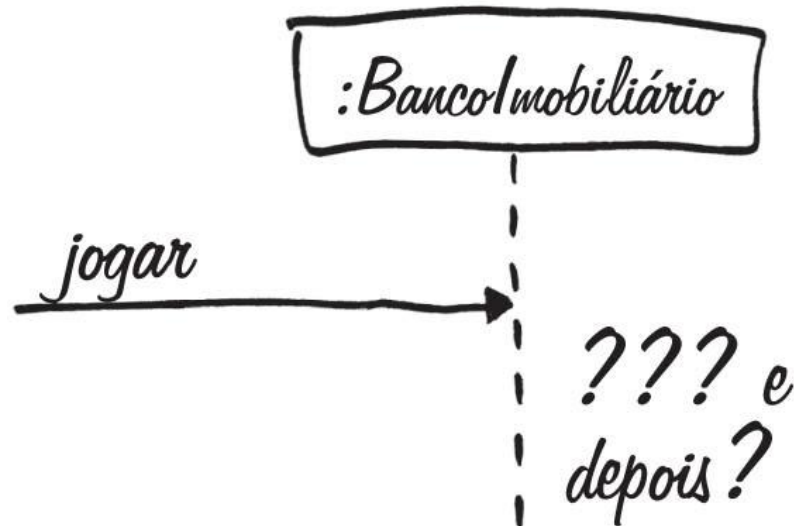
- **Solução (conselho)**

- Atribua **responsabilidades** de modo que a **coesão** permaneça alta.
 - Use isso para **avaliar alternativas**

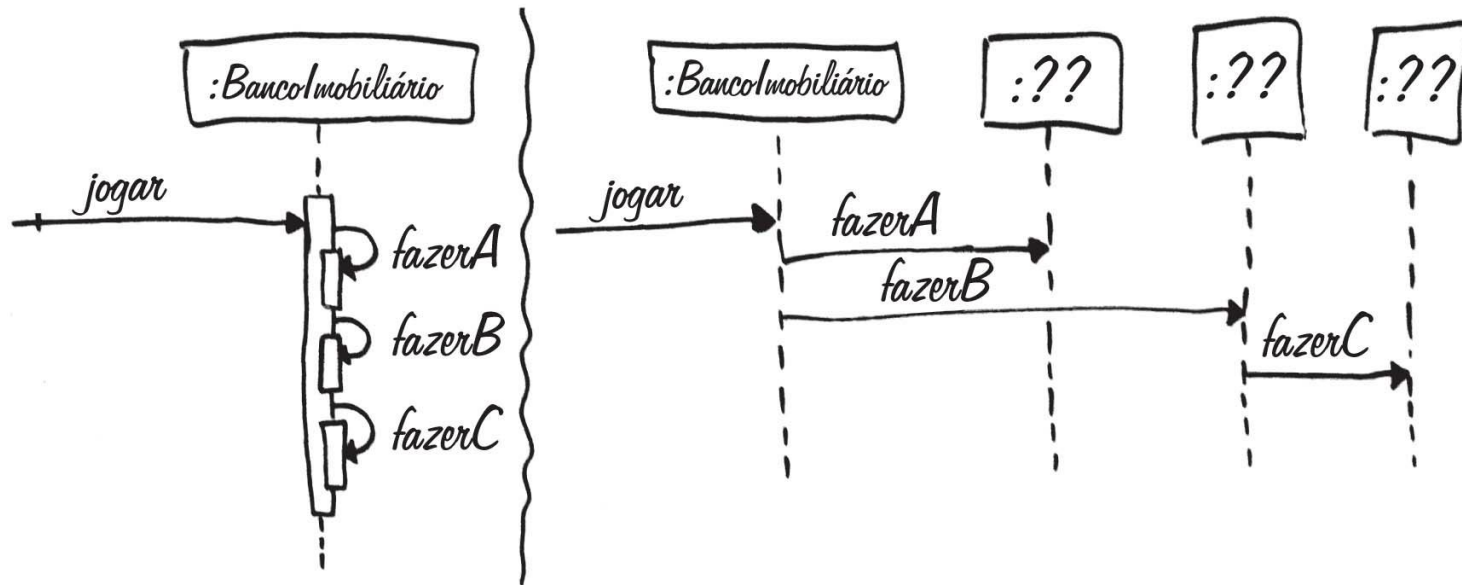
Padrões GRASP

- **Coesão x Acoplamento**
 - Para **minimizar o acoplamento**, conjunto reduzido de **objetos** com **todas as responsabilidades**
 - Objetos **não coesos**
 - Para **maximizar a coesão**, um grande número de objetos com **responsabilidade limitada**
 - Objetos **fortemente acoplados**
 - Escolha a melhor **relação custo benefício** a partir de projetos alternativos

Alternativas de projeto para suporte à Alta Coesão



Alternativas de projeto para suporte à Alta Coesão



*Coesão ruim (baixa)
no objeto BancoImobiliário*

Melhor

Padrões GRASP

***Aplicação dos padrões GRASP ao estudo de caso do
PDV***

Padrões GRASP

- **Criador**
 - **Problema:**
 - Quem deve **criar** objetos **LinhaDelItemVenda**?
 - **Solução (conselho)**
 - Permita que **Venda** o faça
 - **Venda** contém **LinhaDelItemVenda** conforme o **Modelo de Domínio**

Modelo de Domínio parcial

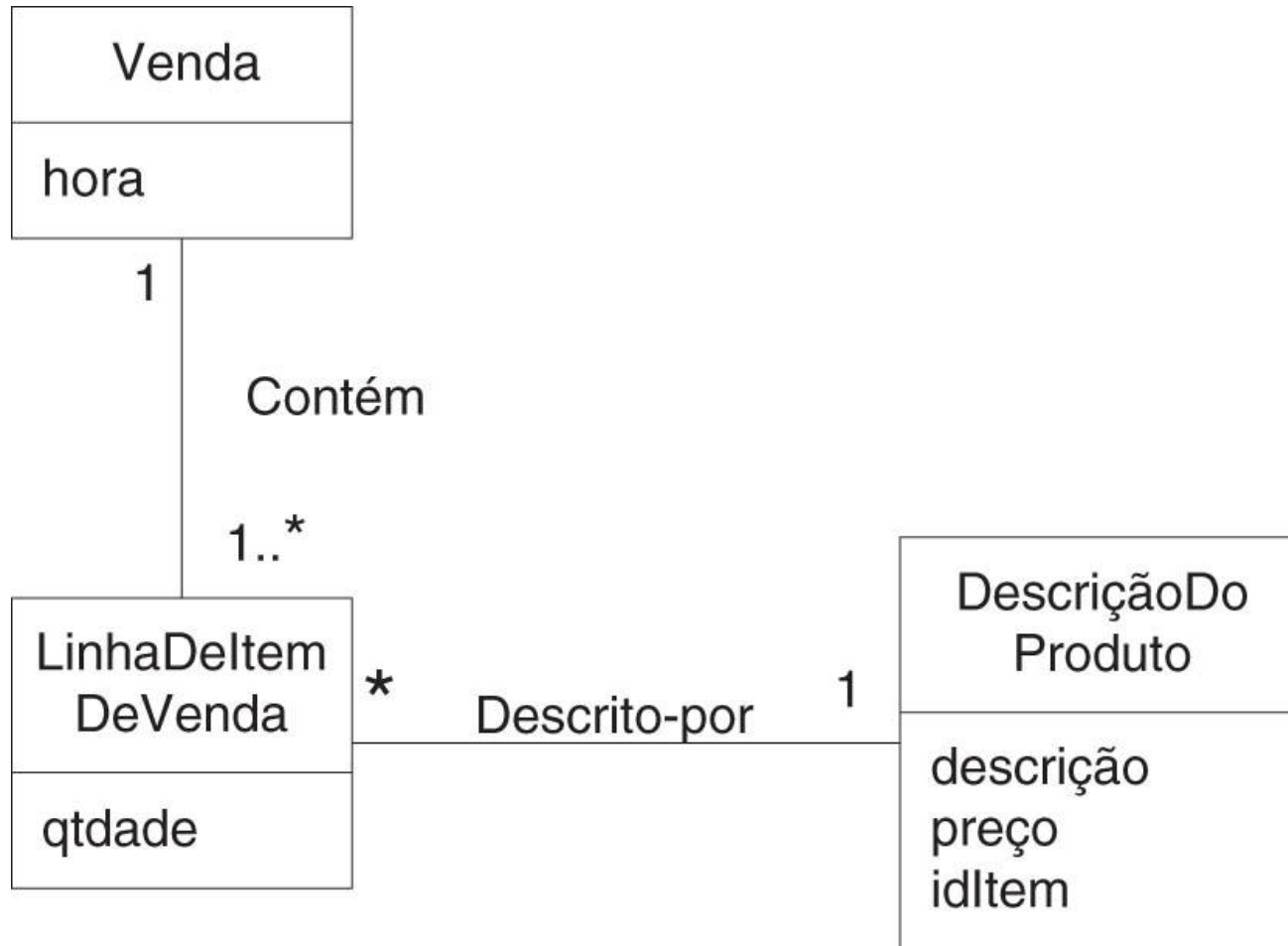
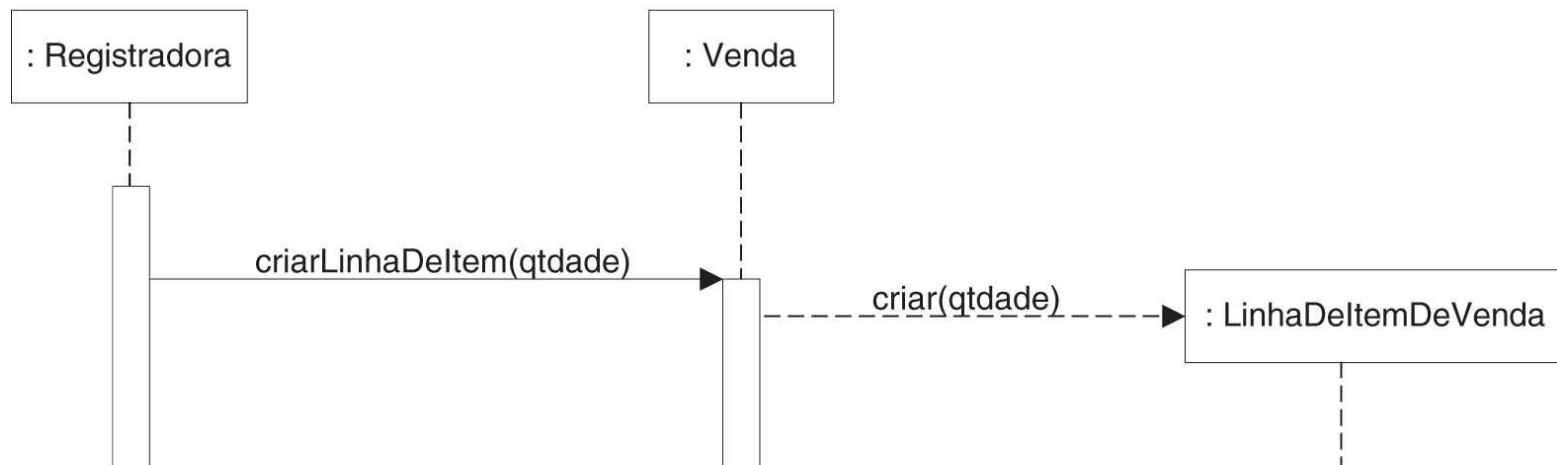


Diagrama de Sequencia parcial

- Precisamos de um método ***criarLinhaDeltemDeVenda(quantidade)*** em ***Venda***
 - Não aumentamos o acoplamento pois ***Venda*** já possui visibilidade para ***LinhaDeltemVenda***



Padrões GRASP

- **Especialista**
 - Quem deve ser o responsável por **saber** o **total** de uma **Venda**?
 - **Solução (conselho)**
 - Permita que **Venda** seja responsável já que ela sabe de todos os **itens da venda**.

Modelo de Domínio parcial

- **Venda** pode calcular seu total solicitando o **subtotal** de cada **LinhaDeItemVenda**

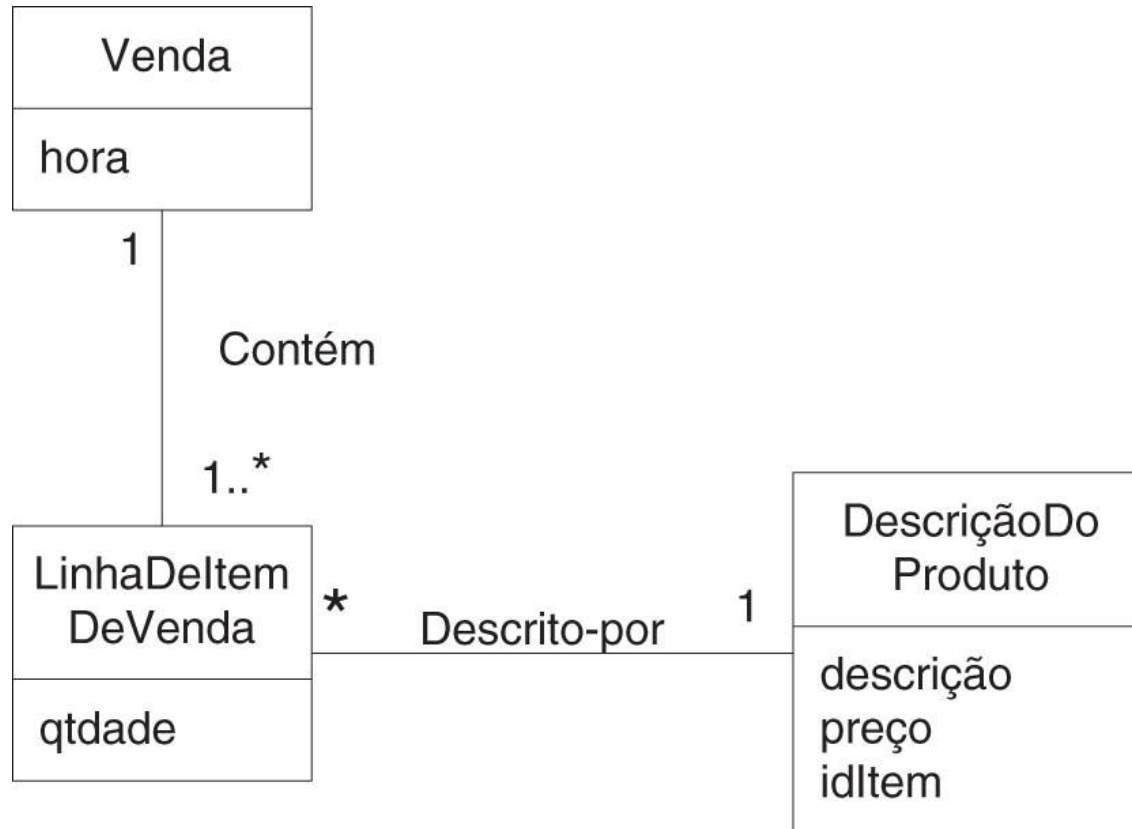
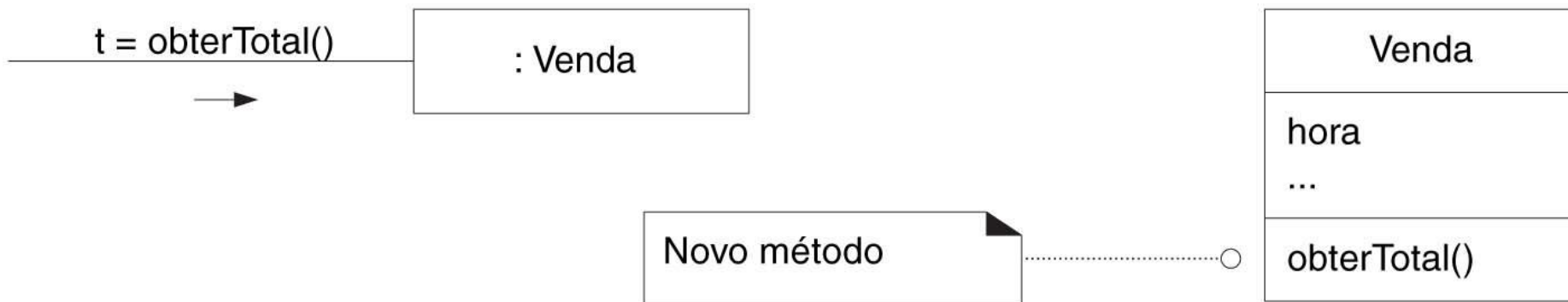


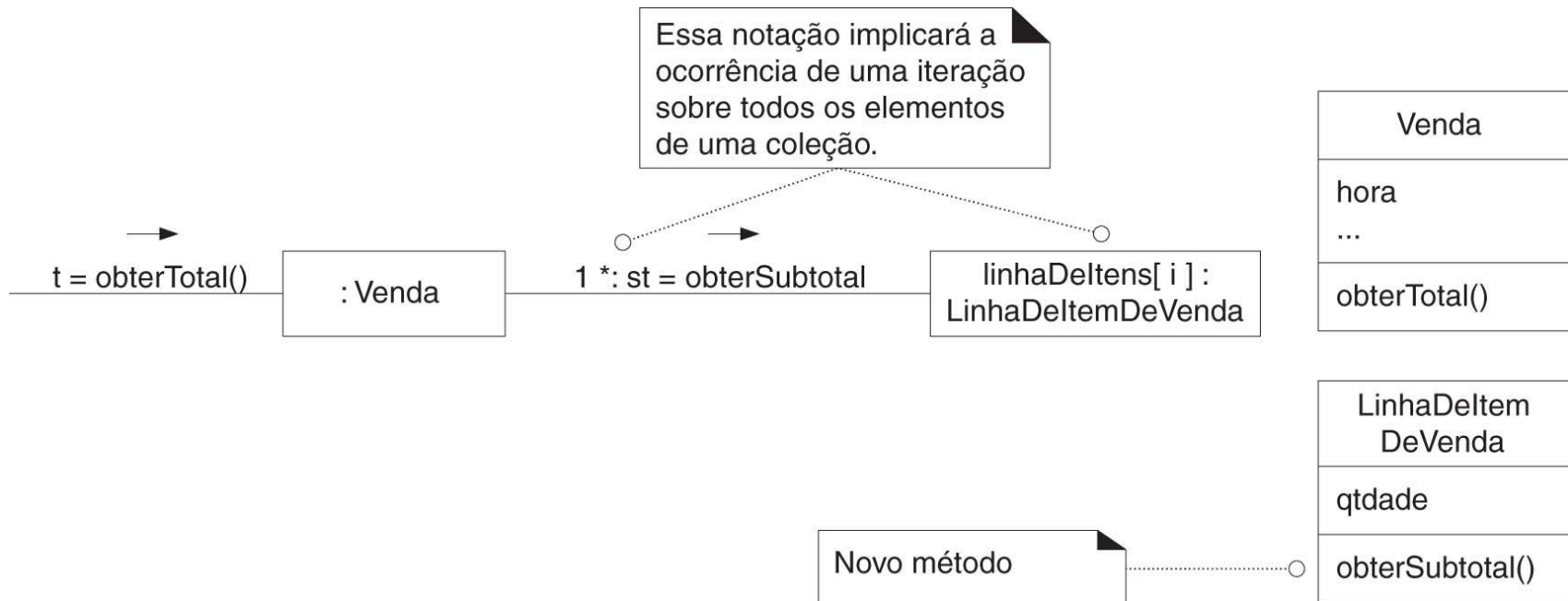
Diagrama de Seqüência Parcial

- Adicione o método *obterTotal()* à *Venda*



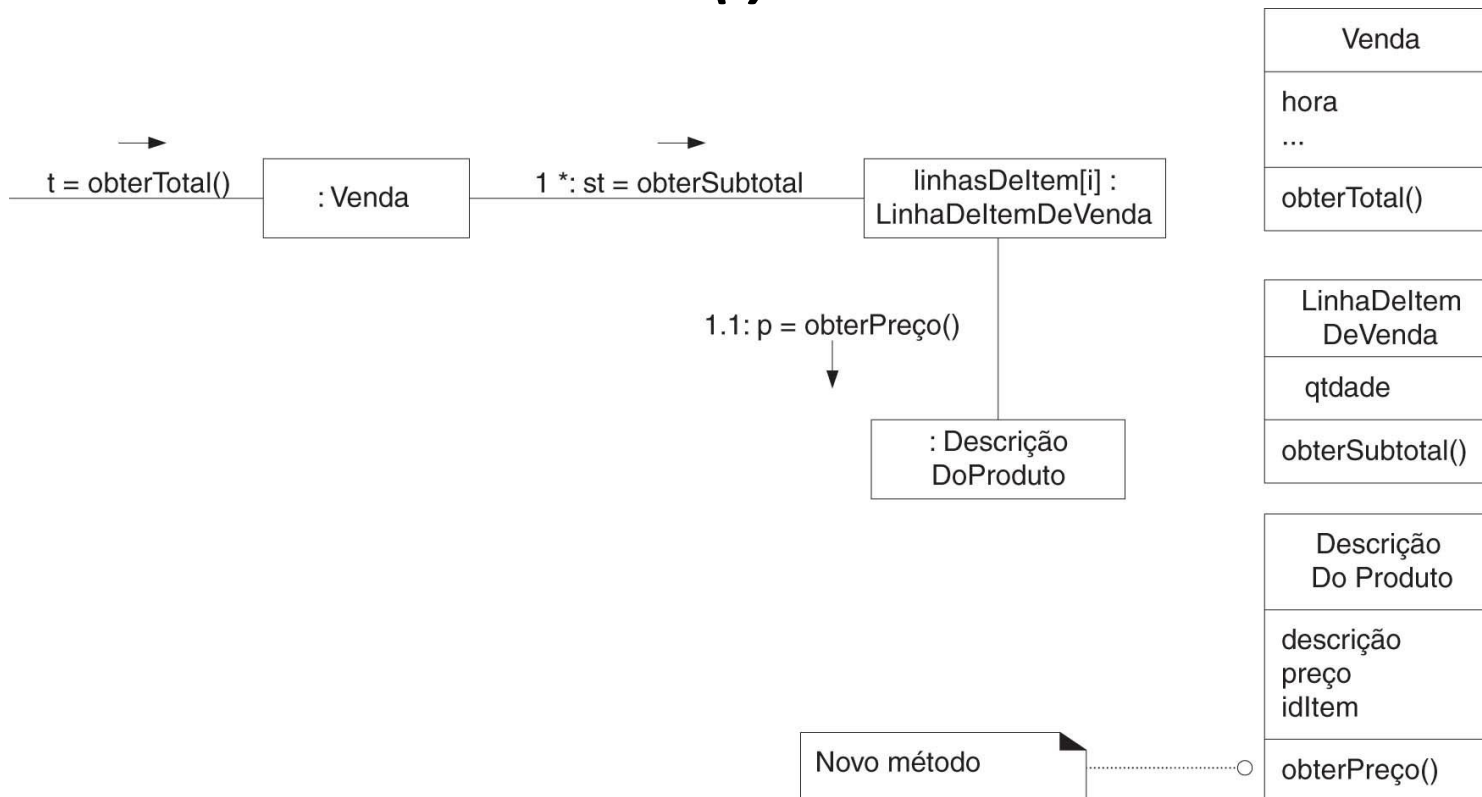
Iterando por itens de venda

- Venda precisa do **subtotal** de cada **item de venda**
 - Crie o método ***getSubtotal()*** em ***LinhaDeltemVenda***



Subtotal necessita do preço unitário

- De acordo com o Especialista, **DescricaoDoProduto** sabe o preço unitário.
 - Adicione ***obterPreco()*** na classe ***DescricaoDoProduto***



Padrões GRASP

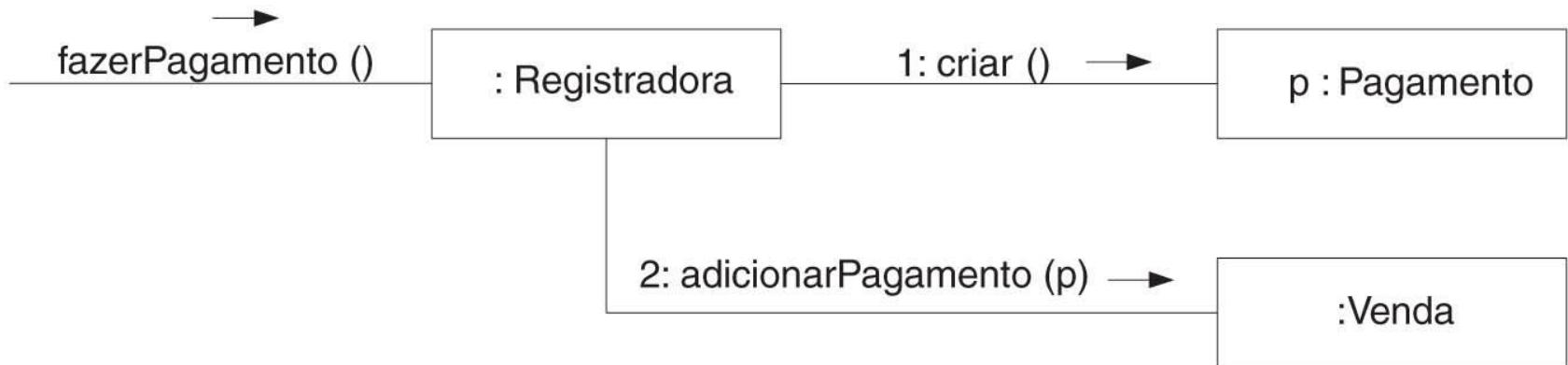
- **Especialista**
 - Um dos **padrões** mais comuns
 - Normalmente, um conjunto de pequenos especialistas se **colaboram**
- Por exemplo, para obter o **total de uma venda**
 - **Venda** obtêm o **subtotal** de **cada item da venda** que por sua vez obtêm o **preço** a partir da **descrição do produto**.

Padrões GRASP

- **Baixo Acoplamento**
 - **Problema:**
 - Quem deve criar um **pagamento** e associá-lo com uma **Venda**?
 - **Alternativas:**
 - Pelo **Especialista**, **Registradora** deveria criar as **informações** de **Pagamento** e associá-las com a **Venda**
 - **Registradora** e **Venda** ficam acopladas à **Pagamento**
 - Por **Baixo Acoplamento**, **Registradora** delega à **Venda** a criação de um **Pagamento**.
 - Apenas **Venda** fica acoplada à **Pagamento**.

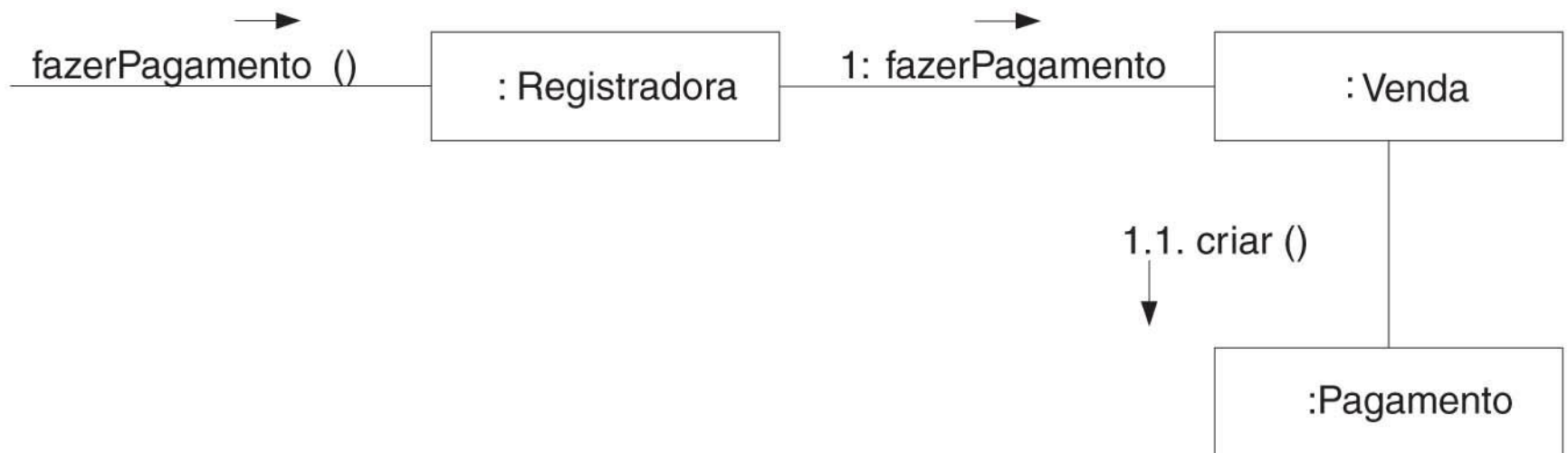
Primeira alternativa

- **Registradora** cria **Pagamento** e o adiciona à **Venda**.
 - **Registradora** e **Venda** ficam acopladas à **Pagamento**.



Segunda Alternativa

- **Registradora** delega a criação de **Pagamento** à **Venda**
 - Pelo **Criador**, **Venda** cria **Pagamento**
 - Apenas **Venda** fica acoplada a **Pagamento**. Um relacionamento necessário



Padrões GRASP

- O **acoplamento** cria **dependências**
- Acoplamentos típicos (Tipos **X** e **Y**)
 - **X** possui um **atributo** do tipo **Y**
 - **X** tem um **método** que referencia **Y**
 - **Parâmetro** no método do tipo **Y**
 - **Variável local** do tipo **Y**
 - **Retorno** do método com tipo **Y**
 - **X** é uma **subclasse** direta ou indireta de **Y**
 - **X** implementa uma **interface** **Y**

Padrões GRASP

- As **classes** se beneficiam do **Baixo Acoplamento**, pois ficam:
 - Mais **independentes**
 - Mais fáceis de serem **reutilizadas**
 - Mais fácil de serem **entendidas**
 - Mais fácil de realizar **manutenções**

Padrões GRASP

- **Baixo Acoplamento**

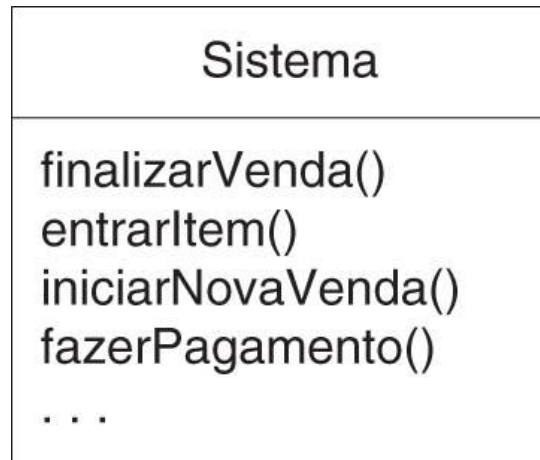
- Levado ao extremo, **poucos objetos** não **coesos** realizam todo o trabalho.
- A severidade do **acoplamento** depende da **instabilidade** do objeto a que está acoplado
 - Classes de **bibliotecas Java** são **estáveis**, portanto o acoplamento a elas é aceitável.
 - **APIs proprietárias** são normalmente **instáveis**, portanto o **acoplamento** pode levar a ficar preso ao **fabricante**.

Padrões GRASP

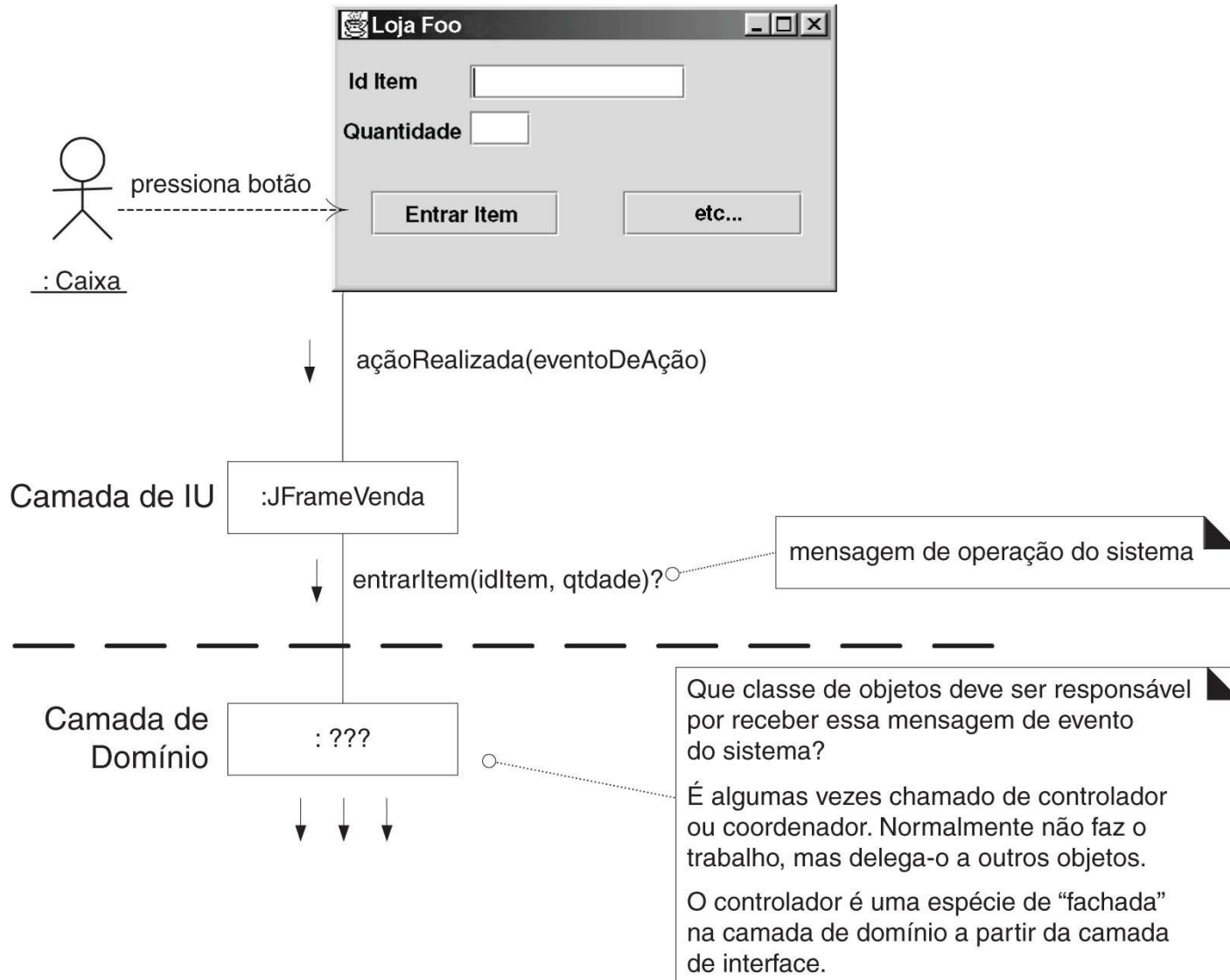
- **Controlador**
 - **Problema:**
 - Qual objeto da **camada de domínio** deve ser responsável pelas **operações do sistema**?
 - **Solução (Conselho):**
 - Selecione o **objeto raiz** no **modelo de domínio** do **PDV**
 - **Loja**
 - Selecione um objeto que representa um dispositivo chave
 - **Registradora**
 - Crie um objeto que trata um caso de uso
 - **TratadorProcessarVenda**

Operações do sistema

- **Diagramas de Seqüência do Sistema** são locais temporários para **operações do sistema**.
 - Um **objeto Controlador** deve conter estas **operações**
 - Quem deve ser o **controlador**?

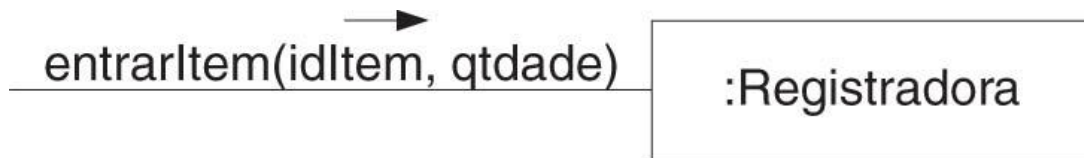


Quem deve ser o controlador?

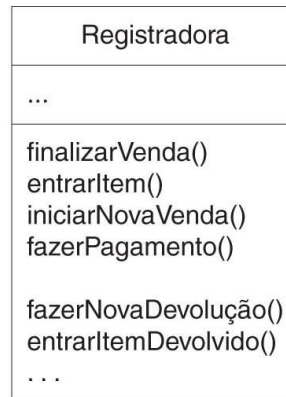
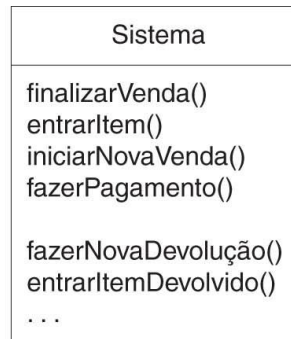


Duas alternativas para o controlador

- ***Registradora*** é um dispositivo chave.
- ***TratadorProcessarVenda*** trata as operações do Sistema para um caso de uso.

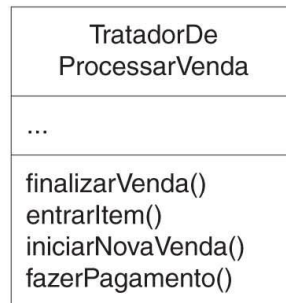
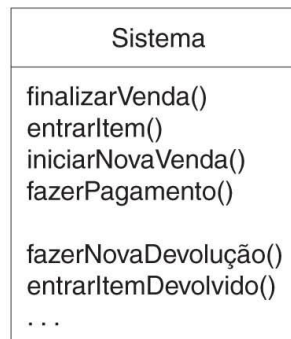


Duas alternativas para o controlador



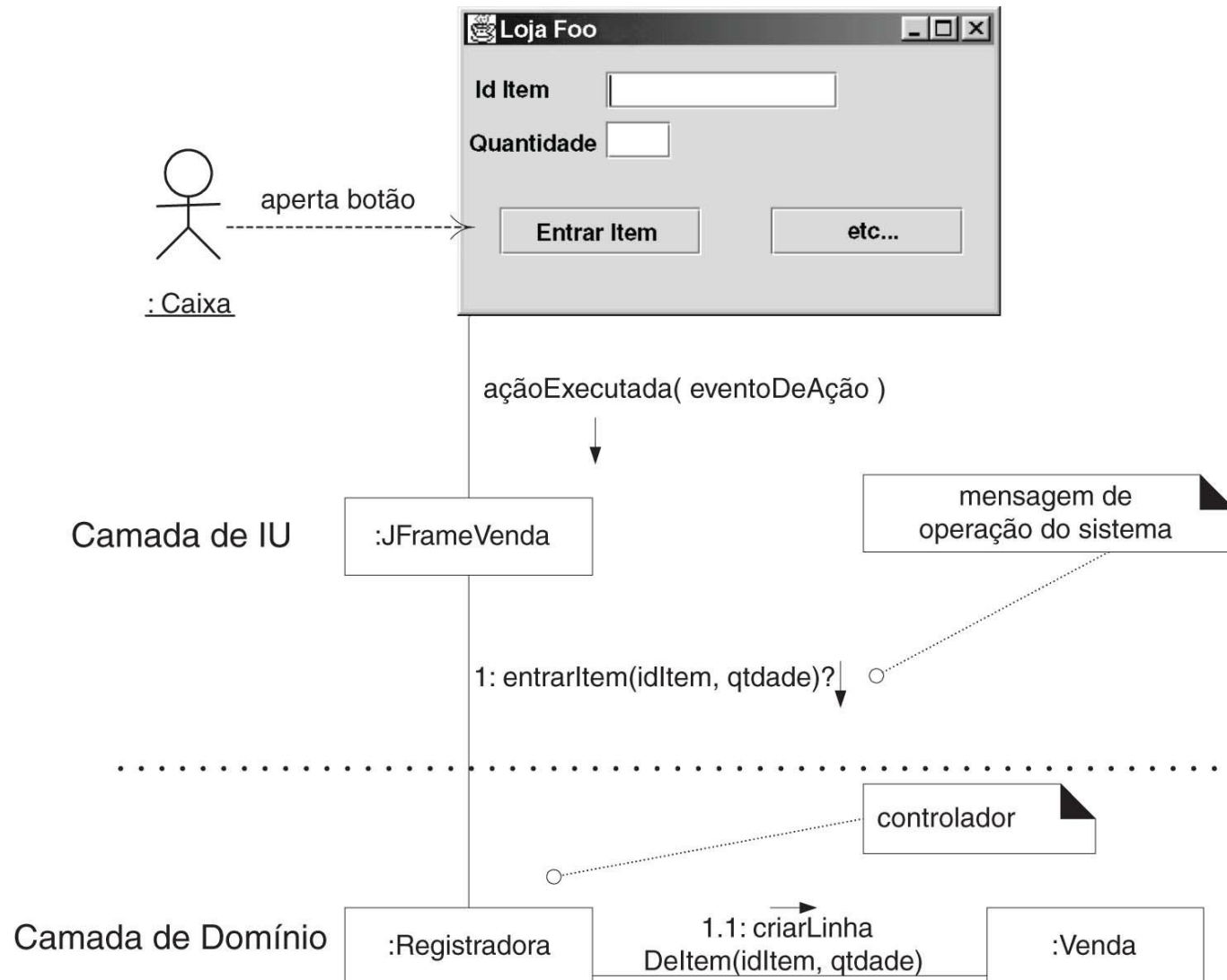
operações do sistema descobertas durante a análise do comportamento do sistema.

alocação de operações do sistema durante o projeto, usando um controlador fachada.



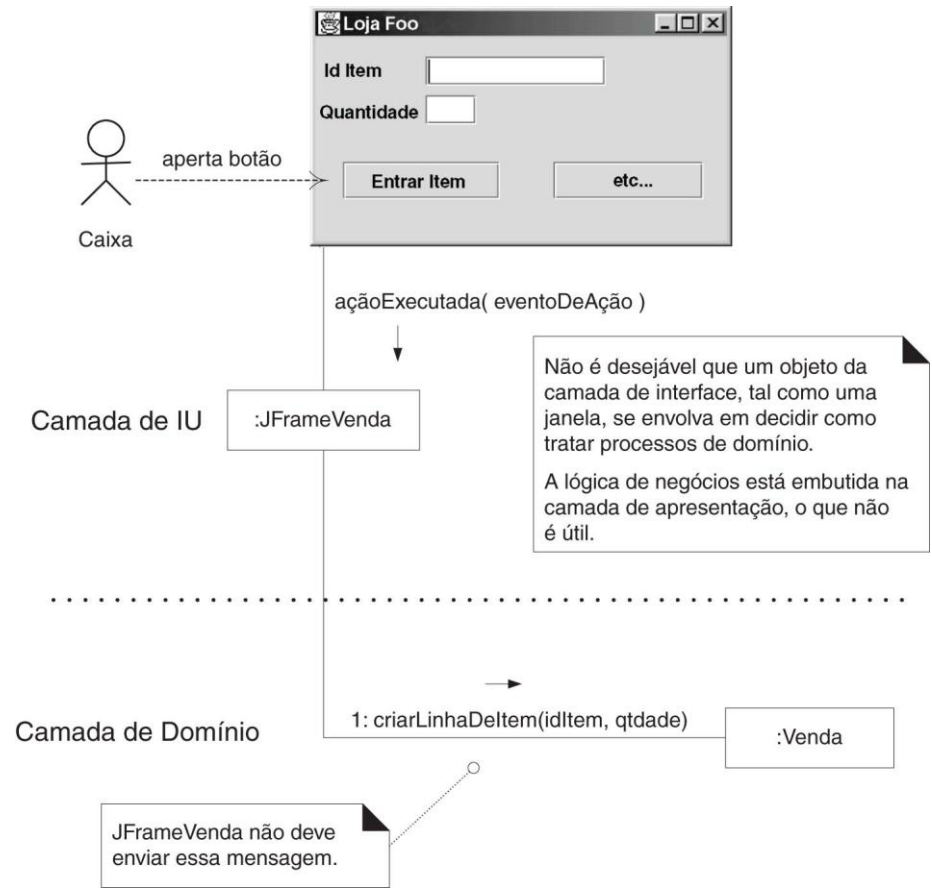
alocação de operações do sistema durante o projeto, usando vários controladores de caso de uso

Registradora como controlador



A GUI enviando mensagem para o objeto Venda

- Este projeto insere mais **lógica de negócios** na **GUI** do que usando ***Registradora*** como **controlador**?



Padrões GRASP

- Questões sobre o **Controlador**
 - Para o **sistema PDV**, por que não usar **Loja** como **Controlador**?
 - Ao usar controladores de **casos de uso**, Larman recomenda um **Tratador** para cada **caso de uso**
 - Por que não permitir um controlador para **casos de uso** relacionados?
 - Um **controlador** essencialmente delega tarefas para **objetos do domínio**
 - **Gerentes**, não trabalhadores
 - Eles mantêm o **estado do caso de uso**
 - “Onde nós estamos..”

Padrões GRASP

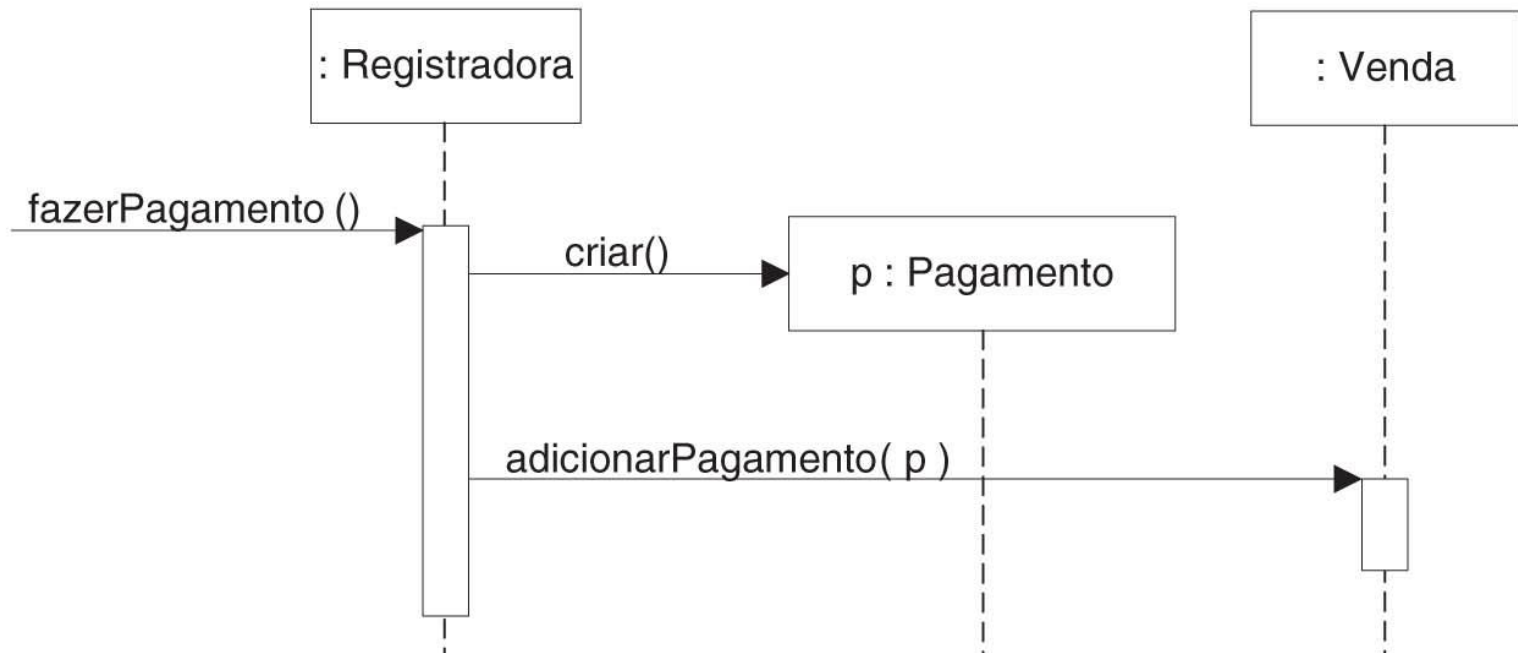
- Questões sobre o **controlador (cont.)**
 - Não atribua **muitas responsabilidades** a um único **controlador**.
 - Lembre-se da **Coesão** !
 - Tornam **GUIs** mais **plugáveis** e **reutilizáveis**
 - Ajudam a manter o **estado** de um **caso de uso**
- Exemplos de **código** de **controladores** para **clientes gordo** (desktop) e **magro** (web)
 - Páginas **324-326**

Padrões GRASP

- **Coesão Alta**
 - **Problema:**
 - Quem dever ser responsável por criar um ***Pagamento***?
 - **Solução (Conselho):**
 - Pelo **Criador**, **Registradora** deveria ser a escolha já que ela possui a informação a respeito
 - **Registradora** pode perder coesão se ela tiver muitas operações
 - Permita que registradora delegue para ***Venda*** a responsabilidade

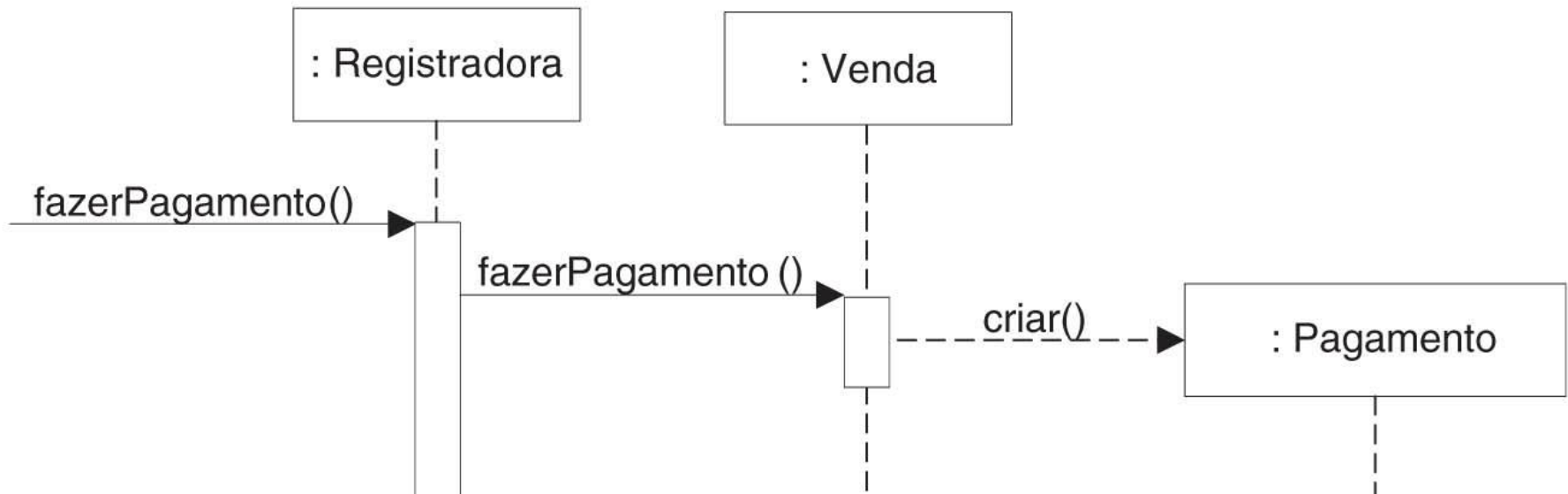
Registradora criando Pagamento

- **Baixa Coesão**
 - ***Registradora*** deve adicionar ***Pagamento*** à ***Venda***



Registradora delegando a criação de Pagamento

- **Alta coesão e menor acoplamento**



Padrões GRASP

- **Níveis de Coesão:**
 - **Muito baixa:** Classe com muitas responsabilidades não relacionadas.
 - **Baixa:** Responsável por uma tarefa complexa em determinada área.
 - **Moderada:** Responsabilidades exclusivas em algumas áreas logicamente relacionadas ao conceito da **classe**, mas não umas com as outras.
 - **Alta:** Responsabilidades moderadas em uma área e colaboração com outras classes.

Padrões GRASP

- **Coesão**
 - **Classes coesas** tendem a possuir **poucos métodos**
 - **Coesão** é muito similar a **programação modular** em **linguagens não orientada a objetos**.